

TP2 – File Integrity Checker and IOC Matcher in Rust

Module 7.1 – Programming with Rust

S. Biha

2025–2026

1. General information

- **Module:** Programming with Rust.
- **TP:** TP2 – File Integrity Checker and IOC Matcher in Rust.
- **Duration:** 3 hours lab work.
- **Mode:** Individual work.
- **Main deliverables:** GitHub repository and PDF technical report.
- **Execution environment:** Docker Compose environment provided for the Rust labs.

2. Scenario

You are part of a defensive security team. The team receives a list of suspicious file hashes, also called indicators of compromise, or IOCs. Your task is to build a small Rust command-line tool that scans a file or a directory, calculates the SHA-256 hash of each file, compares the result with the local IOC list, and generates a simple report.

The objective is not to build a complete antivirus. The objective is to practice secure systems programming in Rust using a realistic defensive-security workflow:

- reading files safely;
- walking through a directory;
- calculating cryptographic hashes;
- loading and validating an IOC file;
- comparing calculated hashes with known suspicious hashes;
- producing a reproducible report;
- handling errors without crashing.

3. Learning objectives

At the end of this TP, the student should be able to:

1. create and run a Rust project inside the provided Docker Compose environment;
2. use Cargo dependencies in a controlled way;

3. calculate SHA-256 hashes of files;
4. parse a simple IOC file format;
5. use `Result`, `Option`, structs, vectors, and hash maps;
6. walk through files and directories;
7. write unit tests for helper functions;
8. produce reproducible execution evidence in a PDF report.

4. Academic integrity and acceptable use

This TP is a defensive programming assignment. Students must only scan local files created for this lab or files explicitly authorized by the teacher.

Students may use course slides, TD material, the Rust official documentation, compiler messages, and the teacher's explanations during the supervised session. Students must not submit code that they cannot explain.

During evaluation, the teacher may ask the student to explain how the SHA-256 hash is calculated, explain how the IOC file is parsed, modify the IOC format slightly, add a new test case, run the tool on a different directory, or show the Git history and Docker execution evidence.

5. Required Docker Compose environment

All commands must be executed using the provided Docker Compose environment.

From the directory containing `docker-compose.yml`, start the environment:

```
mkdir -p workspace
docker compose up -d --build
docker compose exec rustlab bash
```

Inside the container, go to the shared workspace:

```
cd /workspace
```

Check the Rust toolchain:

```
rustc --version
cargo --version
cargo fmt --version
cargo clippy --version
cargo audit --version
cargo deny --version
```

Create the project:

```
cargo new tp2_integrity_checker
cd tp2_integrity_checker
```

Important: all source code must be created inside `/workspace/tp2_integrity_checker`.

6. Expected GitHub repository

The final GitHub repository must contain at least:

```
tp2_integrity_checker/  
|-- Cargo.toml  
|-- README.md  
|-- src/  
|   |-- main.rs  
|   |-- hashing.rs  
|   |-- ioc.rs  
|   |-- scanner.rs  
|   |-- report.rs  
|-- samples/  
|   |-- files/  
|   |   |-- clean_readme.txt  
|   |   |-- suspicious_dropper.txt  
|   |   |-- notes.txt  
|   |-- iocs.txt  
|-- reports/  
|   |-- scan_report.csv  
|-- screenshots/  
|   |-- 01_environment.png  
|   |-- 02_cargo_run.png  
|   |-- 03_scan_result.png  
|   |-- 04_cargo_test.png  
|   |-- 05_cargo_clippy.png  
|-- report/  
|   |-- TP2_Report.pdf
```

The exact internal module names may differ, but the repository must remain clean, readable, and reproducible.

7. Required dependency

The Rust standard library does not provide SHA-256 hashing. For this TP, students may use the `sha2` crate.

Edit `Cargo.toml`:

```
[package]  
name = "tp2_integrity_checker"  
version = "0.1.0"  
edition = "2021"  
  
[dependencies]  
sha2 = "0.10"
```

Then check that the project still builds:

```
cargo check
```

8. Sample input data

Create the sample directory:

```
mkdir -p samples/files
```

Create three sample files:

```
printf "Welcome to TP2 Rust integrity lab.\n" > samples/files/clean_readme.txt  
printf "MALWARE-TEST-SAMPLE-TP2\n" > samples/files/suspicious_dropper.txt  
printf "Normal administrative notes.\n" > samples/files/notes.txt
```

Check the hashes manually with the Linux tool:

```
sha256sum samples/files/*
```

Expected hashes:

```
70bbeaa0f2d408a45827aa9d5bd58209564a5bd7b61d6c069267c9e9e35f97cd  samples/files/
clean_readme.txt
1888673b4c962129e57b54b81fda2f967c52f87c28e9d7908e5fba0dfed097e3  samples/files/notes.
txt
44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b  samples/files/
suspicious_dropper.txt
```

Create the IOC file:

```
nano samples/iocs.txt
```

Use the following content:

```
# hash,label

44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b,Demo suspicious test
sample
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa,Fake known malware
hash
INVALID_HASH_LINE_SHOULD_BE_IGNORED
```

The invalid IOC line must not crash the program.

9. Functional requirements

Your program must be executable with:

```
cargo run -- --target samples/files --ioc samples/iocs.txt --report reports/scan_report
.csv
```

The program must:

1. accept a target file or directory using `-target`;
2. accept an IOC file using `-ioc`;
3. accept an output report path using `-report`;
4. calculate SHA-256 for each regular file;
5. compare each calculated hash with the IOC list;
6. display a clear summary in the terminal;
7. generate a CSV report;
8. ignore empty lines and comment lines in the IOC file;
9. ignore invalid IOC lines but count them;
10. handle missing files or directories with a clean error message;
11. avoid uncontrolled `unwrap()` and `expect()` in the main execution path.

10. Suggested command-line interface

Minimum required usage:

```
cargo run -- --target <FILE_OR_DIRECTORY> --ioc <IOC_FILE> --report <REPORT_FILE>
```

Example:

```
cargo run -- --target samples/files --ioc samples/iocs.txt --report reports/scan_report.csv
```

If arguments are missing, the program should print:

```
Usage:
tp2_integrity_checker --target <FILE_OR_DIRECTORY> --ioc <IOC_FILE> --report <REPORT_FILE>
```

11. Suggested terminal output

Your exact output may differ, but it must contain the same information.

```
TP2 File Integrity Checker and IOC Matcher
Target: samples/files
IOC file: samples/iocs.txt
Report: reports/scan_report.csv

Summary:

* Files scanned: 3
* IOC entries loaded: 2
* Invalid IOC lines: 1
* Matches found: 1
* Errors: 0

Matches:
[ALERT] samples/files/suspicious_dropper.txt
SHA-256: 44ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b
IOC label: Demo suspicious test sample

CSV report written to reports/scan_report.csv
```

12. Expected CSV report format

The generated CSV report must contain at least the following columns:

```
path,sha256,status,label
samples/files/clean_readme.txt,70
bbeaa0f2d408a45827aa9d5bd58209564a5bd7b61d6c069267c9e9e35f97cd,CLEAN,
samples/files/notes.txt,1888673
b4c962129e57b54b81fda2f967c52f87c28e9d7908e5fba0dfed097e3,CLEAN,
samples/files/suspicious_dropper.txt,44
ea92bec1f9e8aa690d8aceddf1294e9fb4a71d39769d6f383e3915ac76bb3b,MATCH,Demo suspicious
test sample
```

13. Technical constraints

The following constraints are mandatory:

- the program must be written in Rust;
- the program must run inside the Docker Compose container;
- the main execution path must not use uncontrolled `unwrap()` or `expect()`;
- the program must not use `unsafe`;
- the program must use SHA-256, not MD5 or SHA-1;
- invalid IOC lines must not stop execution;
- missing files must produce meaningful error messages;
- the code must be split into meaningful functions or modules;
- the project must contain at least four unit tests;
- `cargo fmt`, `cargo clippy`, and `cargo test` must pass.

14. Recommended internal design

You are encouraged to use a data model similar to the following.

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct IocEntry {
    pub hash: String,
    pub label: String,
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ScanStatus {
    Clean,
    Match(String),
    Error(String),
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct ScanResult {
    pub path: String,
    pub sha256: Option<String>,
    pub status: ScanStatus,
}
```

Suggested helper functions:

```
pub fn hash_file_sha256(path: &std::path::Path) -> Result<String, std::io::Error> {
    // Calculate and return the SHA-256 hash as lowercase hexadecimal.
}

pub fn load_iocs(path: &std::path::Path) -> Result<Vec<IocEntry>, std::io::Error> {
    // Read and parse the IOC file.
}

pub fn is_valid_sha256(value: &str) -> bool {
    // Return true only if the value is 64 hexadecimal characters.
}
```

```
}

pub fn scan_target(
    target: &std::path::Path,
    iocs: &[IoEntry],
) -> Vec<ScanResult> {
    // Scan one file or all regular files in a directory.
}
```

15. PDF report structure

The PDF report must contain the following sections:

1. **Title page:** student name, module name, TP title, GitHub repository URL, and date.
2. **Objective:** short explanation of the tool, why file-integrity checking is useful, and what an IOC is.
3. **Environment:** Docker Compose commands, Rust toolchain version, and project path inside the container.
4. **Implementation:** hashing function, IOC parsing, scanning, CSV report generation, and command-line handling.
5. **Execution evidence:** screenshots of `sha256sum`, `cargo run`, the CSV report, `cargo test`, and `cargo clippy -D warnings`.
6. **Discussion:** invalid IOC handling, missing-file handling, why SHA-256 is preferred over MD5 or SHA-1, and possible improvements.
7. **Conclusion:** summary of the implementation and Rust concepts practiced.

16. Bonus tasks

The following tasks are optional and can improve the grade only if the mandatory requirements are already satisfied:

- implement recursive directory scanning;
- add a `-json` output option;
- add a `-only-matches` option;
- add a known-good manifest mode;
- add simple parallel scanning using Rust threads;
- add integration tests using files under `samples/`;
- add `cargo audit` evidence in the PDF report.

17. Final submission format

Submit:

1. the GitHub repository URL;
2. the PDF report named as follows:

```
TP2_Rust_Integrity_Checker_FirstName_LastName.pdf
```

The deadline and submission platform will be communicated by the teacher.